EXPRESS AUTOCARE

MULTI-BRAND CAR SERVICE CENTER

Express Autocare ERP

App Installation & Play Store Guide

Testing the staff app and the customer app on phones,

then publishing them on Google Play.

Version 1 · October 2026 · for the owner and whoever sets up the system

Contents

1. The two apps 3
What this guide covers 3
Already set up on this computer 3
2. Run the system for phones on the workshop Wi-Fi 5
3. An HTTPS address for testing 6
4. The staff app (EA Workshop): build and install 7
4.1 Build the test app (APK) 7
4.2 Put it on a phone 7
4.3 First start 7
4.4 Changing the server address 8
4.5 Test checklist (staff app) 8
5. The customer app (Express Autocare): test 10
5.1 As a web app (no build needed) 10
5.2 As an Android app (APK for testing) 10
5.3 Test checklist (customer app) 11
6. Going live: the server 12
7. The signing key (once) 13
Tell the build where the key is 13
8. Release build for Google Play 14
8.1 Before the first upload 14
8.2 Build 14
9. Publishing on the Google Play Store 15
9.1 The developer account 15
9.2 Create the app and the store listing 15
9.3 App access for Google's reviewers 15
9.4 Data safety answers (customer app) 16
9.5 Testing tracks, then production 16
9.6 Updates 16
9.7 The staff app and the Play Store 16
10. Problems and fixes 18

1. The two apps

Express Autocare ERP has two phone apps. Both open the live system, so every change made to the system reaches them at once; a new app file is needed only when the app itself changes (its permissions, icon or name).

	EA Workshop (staff app)	Express Autocare (customer app)
Who	Staff: drivers, home technicians, advisors, managers	Customers
What it opens	The whole system (same as the browser)	The customer app at <address>/app
Why an app	Live location keeps going with the screen locked	An icon on the phone, notifications, Play Store presence
Server address	Typed on the phone the first time (can be changed)	Built into the app (customers never type it)
How staff / customers get it	An APK file you share (no Play Store needed)	Google Play Store (or the web app until then)
Project folder	mobile/	mobile-customer/
App id	in.expressautocare.erp	in.expressautocare.app

Without any app: both also work in the phone's browser and can be added to the home screen (staff: the system address; customers: the address + /app). The only difference for staff is that the browser shares location only while the app is open.

What this guide covers

- **Part A (sections 2–5): testing on your phones before the system is online**: running the system on this computer, an HTTPS address for the phones, building and installing both apps.
- **Part B (sections 6–9): going live and publishing**: the server, the signing key, release builds and the Google Play Store.
- Section 10: problems and fixes.

Already set up on this computer

Tool	Where	Used for
PHP 8.4, Composer	C:\php	Running the system
MariaDB	C:\Program Files\MariaDB 13.0	The database
Node.js	C:\Program Files\nodejs	Building the screens and the apps
JDK 17	C:\Users\JOEL\.jdks\ms-17.0.16	Building Android apps

Tool	Where	Used for
Android SDK	%LOCALAPPDATA%\Android\Sdk	Building Android apps (platforms 34 and 36)

> **Note** Commands in this guide are typed in **Git Bash** (or a Command Prompt where shown with "set"), from the project folder **C:\Users\JOEL\EAERP** unless it says otherwise.

2. Run the system for phones on the workshop Wi-Fi

While the system is not online, it runs on this computer. Phones on the same Wi-Fi can reach it at the computer's address.

1. Build the screens once (phones cannot use the developer preview):

```
npm run build
del public\hot      (Command Prompt)   or   rm -f public/hot   (Git Bash)
```

2. Start the system so the Wi-Fi can reach it (leave this window open):

```
php artisan serve --host 0.0.0.0 --port 8000
```

3. Find the computer's Wi-Fi address: run **ipconfig** and note the **IPv4 Address** of the Wi-Fi adapter, for example 192.168.1.20.
4. The first time, Windows asks whether to allow PHP on the network: choose **Private networks** and **Allow**. (Or allow port 8000 in Windows Defender Firewall › Advanced settings › Inbound rules.)
5. On a phone on the same Wi-Fi, open **http://192.168.1.20:8000** in Chrome. The sign-in screen should appear.

> **Careful** Also run the scheduler while testing (reminders, live-location clean-up, WhatsApp sending): open a second window and run **php artisan schedule:work**.

> **Note** When you go back to programming on this computer, **npm run dev** recreates public\hot and the phones stop working until you build again.

3. An HTTPS address for testing

Phone browsers allow **location, the camera, installing as an app and notifications only on https:// addresses** (or on the phone itself). A plain http://192.168.… address is enough to look around, but punching in, live location and the customer app's notifications will not work. A free **Cloudflare quick tunnel** gives this computer a temporary https address that works from any phone, even on mobile data.

1. Download **cloudflared** for Windows (64-bit) from Cloudflare's website (developers.cloudflare.com › Cloudflare Tunnel › Downloads) and put **cloudflared.exe** in C:\Users\JOEL\bin.
2. With the system running (section 2), open another window and run:

```
cloudflared tunnel --url http://localhost:8000
```

3. After a few seconds it prints an address like **https://bright-river-1234.trycloudflare.com**. Open it on a phone: the sign-in screen appears, with a padlock.
4. In the system: **Settings › App Address & Customer App › Public address** = that https address, **Save**. Links sent to customers and the staff-app QR code now use it.
5. Set the same address in **.env** as **APP_URL=https://bright-river-1234.trycloudflare.com** and run **php artisan config:clear**, so photos and files open over https too.

> **Careful** A quick tunnel gets a **new address every time** it starts. Update the public address (and the phones' server address, section 4.4) each time, or create a named Cloudflare tunnel on your own domain for a fixed address. Anyone with the address can reach the sign-in screen: stop the tunnel (Ctrl + C) when you finish testing.

> **Tip** Only for a quick look on one test phone without a tunnel: in Chrome on the phone open **chrome://flags**, search "Insecure origins treated as secure", add http://192.168.1.20:8000, and relaunch Chrome.

4. The staff app (EA Workshop): build and install

4.1 Build the test app (APK)

```
cd C:\Users\JOEL\EAERP\mobile
npm install                    (first time only)
set JAVA_HOME=C:\Users\JOEL\.jdks\ms-17.0.16
npm run build:debug
```

The app file is **mobile\android\app\build\outputs\apk\debug\app-debug.apk** (about 4 MB). Rename it to **EA-Workshop.apk** before sharing.

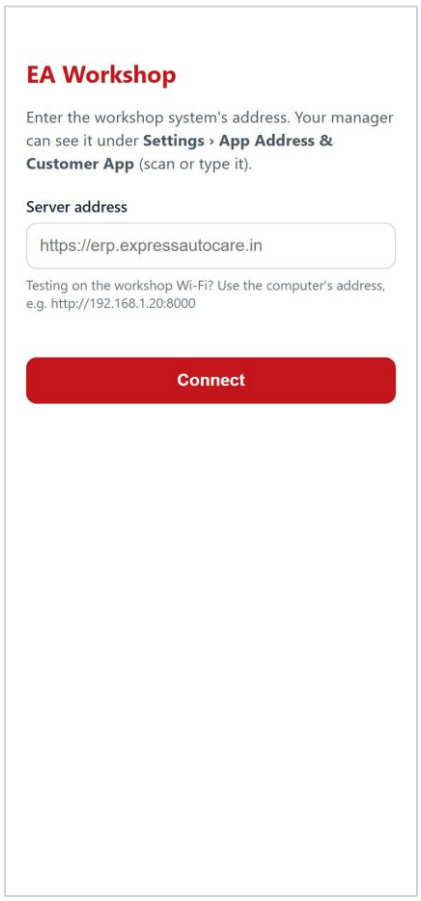
> **Tip** To have the setup screen already filled in, set the address before building: **set EAERP_APP_URL=https://bright-river-1234.trycloudflare.com**. Staff can still change it.

4.2 Put it on a phone

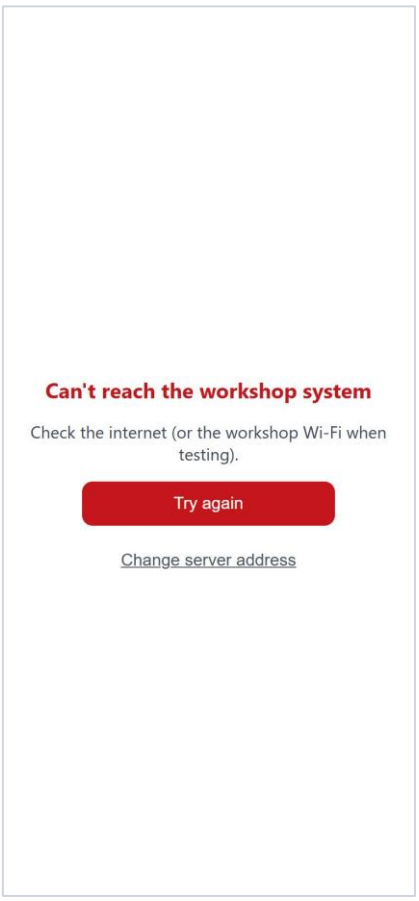
1. Send **EA-Workshop.apk** to the phone (WhatsApp to yourself, Google Drive, or a USB cable into the phone's Download folder).
2. On the phone, tap the file. If Android says it is blocked, tap **Settings** and allow **Install unknown apps** for WhatsApp / Files / Chrome (whichever opened it), then go back and tap **Install**.
3. If Google Play Protect warns about an unknown developer, tap **More details › Install anyway** (this is normal for apps not from the Play Store).

4.3 First start

1. Open **EA Workshop**. Type the server address (the https tunnel address, or later the live address) and tap **Connect**.
2. Sign in with the phone number or email and password.
3. Punch in: allow **Camera** (selfie), **Location → While using the app** and **Notifications**, then tap **I understand, continue** on the location notice.
4. Phone Settings › Apps › EA Workshop › **Battery › Unrestricted** (on Xiaomi / Redmi also **Autostart: on**; on Oppo / Vivo / Realme "Allow background activity").



First start: the server address



Server not reachable

4.4 Changing the server address

In the app: **Settings** (from the menu) › **Change server address** (tap twice), or on the "Can't reach the workshop system" screen tap **Change server address**. The setup screen opens again.

4.5 Test checklist (staff app)

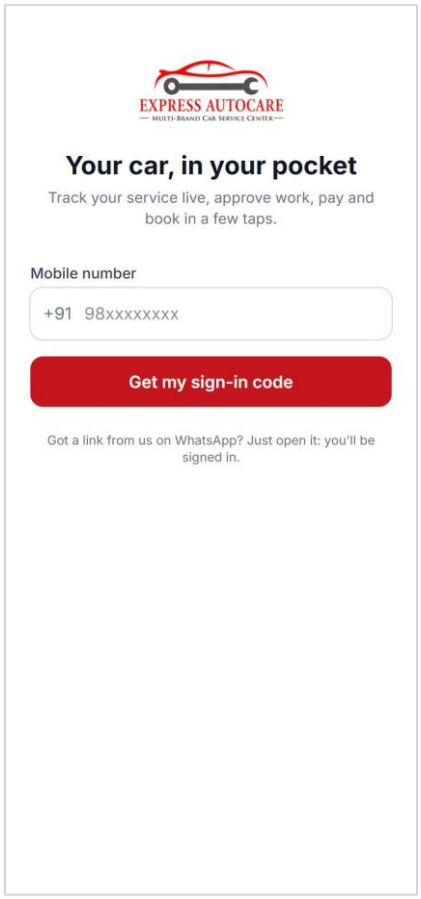
Check	How	Expected
Sign-in	Sign in as a driver	My Day / My Trips open
Punch with location	Punch in at the workshop	Green line "Sharing your location until you punch out"
Background location	Lock the phone, walk or drive 10 minutes	Owner's Live Staff Map shows the route; battery and "Android app" under the name
Notification	Pull down the notification shade	"EA Workshop is sharing your location"
Punch out	Punch out	Sharing stops; the map shows "Punched out"

Check	How	Expected
Trip	Start a pickup assigned to you	The customer's job page shows you on the way

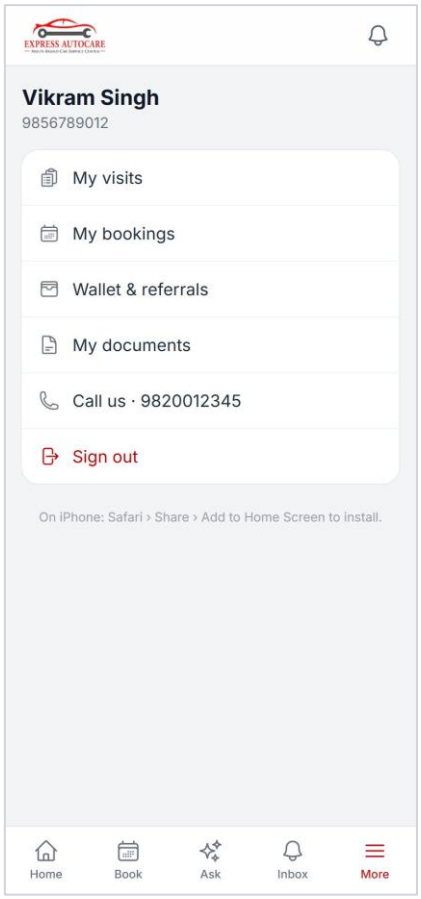
5. The customer app (Express Autocare): test

5.1 As a web app (no build needed)

1. Open **<https address>/app** on a phone, or scan the customer QR on Settings › App Address & Customer App.
2. Sign in: open a job link sent from a test job (Open in the Express Autocare app), or enter the customer's number and read the code from their page in the system.
3. Install: Chrome menu › **Install app** (Android) or Safari › Share › **Add to Home Screen** (iPhone). Allow notifications when Inbox offers it.



Sign-in



More › Install the app

5.2 As an Android app (APK for testing)

The customer app has the address built in, so build it with the address the test phones will use:

```
cd C:\Users\JOEL\EAERP\mobile-customer
npm install                      (first time only)
set EAERP_APP_URL=https://bright-river-1234.trycloudflare.com
set JAVA_HOME=C:\Users\JOEL\.jdks\ms-17.0.16
npm run build:debug
```

Install **mobile-customer\android\app\build\outputs\apk\debug\app-debug.apk** the same way as the staff app (4.2). It opens straight on the customer sign-in screen.

> **Careful** A test APK made with a tunnel address stops working when the tunnel address changes: rebuild it. The Play Store version is built once with the live address (section 8).

5.3 Test checklist (customer app)

Check	Expected
Open a WhatsApp job link › Open in the app	Signed in, that job opens
Sign in with number + code (read out by staff)	Home with their jobs and cars
Approve / decline an item	The advisor sees the answer on the estimate
Pay by UPI › send the reference	Back office sees it in UPI Payments to Confirm; after Confirm the app shows "Received"
Book a service	Appears in CRM › Bookings as "Booked online"
Add a document with an expiry	Shows in Documents and on the customer's page
Driver starts a pickup	The job shows the driver on the map
Inbox › turn on notifications, then move the job to a new stage	A phone notification arrives (needs https)

6. Going live: the server

Before publishing the customer app, the system must be online on its own address with HTTPS. A small cloud server (VPS) with 2 CPUs, 4 GB RAM and 40 GB disk is enough to start.

- **Domain:** for example erp.expressautocare.in, pointing to the server.
- **Software:** PHP 8.4 (with gd, mbstring, openssl, bcmath, curl, zip, pdo_mysql), MariaDB, a web server (nginx), Node.js only for building.
- **HTTPS certificate:** free from Let's Encrypt (certbot), renewed automatically.
- **.env:** APP_ENV=production, APP_DEBUG=false, APP_URL=https://erp.expressautocare.in, the database details.
- **Scheduler:** a cron entry running **php artisan schedule:run** every minute (reminders, location clean-up, WhatsApp, attendance).
- **Backups:** a nightly database dump and the storage folder (photos, documents) copied off the server.

1. Copy the project to the server, then run: **composer install --no-dev**, **npm ci && npm run build**, **php artisan migrate --force**, **php artisan storage:link**, **php artisan config:cache**.
2. Sign in as the owner: **Settings › App Address & Customer App** = https://erp.expressautocare.in; UPI ID; helpline.
3. On staff phones: **Change server address** to the live address (4.4).

> **Note** Moving the data from this computer: dump the database here (mariadb-dump) and load it on the server, and copy storage\app (photos, selfies, documents).

7. The signing key (once)

Every Android app is signed. Google Play keeps its own key for the app; you sign each upload with your **upload key**. Make it once and keep it safe: without it you cannot publish updates (Google can reset it, but it takes days).

```
cd C:\Users\JOEL
"C:\Users\JOEL\.jdks\ms-17.0.16\bin\keytool" -genkey -v -keystore ea-upload.jks
    -keyalg RSA -keysize 2048 -validity 10000 -alias ea-upload
```

It asks for a password (twice), your name and organisation. Then:

- Keep **ea-upload.jks** and its password in two safe places (for example a password manager and an encrypted USB drive). **Never** put it in the project folder or on GitHub.
- Use the same key for both apps, or make a second one for the staff app.

Tell the build where the key is

Create **mobile-customer\android\keystore.properties** (not shared, not committed):

```
storeFile=C:/Users/JOEL/ea-upload.jks
storePassword=<your password>
keyAlias=ea-upload
keyPassword=<your password>
```

Then in **mobile-customer\android\app\build.gradle**, inside **android { … }**, add a signing configuration that reads it and use it for the release build (or use Android Studio: **Build › Generate Signed App Bundle** and pick the key; it does the same).

```
def ks = new Properties(); ks.load(new
FileInputStream(rootProject.file("keystore.properties")))
signingConfigs { release { storeFile file(ks.storeFile); storePassword ks.storePassword
                           keyAlias ks.keyAlias; keyPassword ks.keyPassword } }
buildTypes { release { signingConfig signingConfigs.release; minifyEnabled false } }
```

8. Release build for Google Play

8.1 Before the first upload

- **Target Android version:** Google Play accepts new apps only if they target a recent Android version (API 35 or newer at the time of writing; Play Console shows the current rule). The projects currently target API 34 (Capacitor 6). Raise **targetSdkVersion** and **compileSdkVersion** in **android\variables.gradle** to what Play requires (this computer has Android 36 installed), or upgrade Capacitor, and test again. Ask for this to be done before the first upload.
- **Version:** in **android\app\build.gradle** set **versionName** (what users see, e.g. "1.0.0") and **versionCode** (a number that must go up with every upload: 1, 2, 3 …).
- **Icon and name:** the icon is the Express Autocare logo already; the name is "Express Autocare".

8.2 Build

```
cd C:\Users\JOEL\EAERP\mobile-customer
set EAERP_APP_URL=https://erp.expressautocare.in
set JAVA_HOME=C:\Users\JOEL\.jdks\ms-17.0.16
npm run build:release
```

The file to upload is **android\app\build\outputs\bundle\release\app-release.aab** (an "Android App Bundle"; Play makes the phone-specific APKs from it).

> **Careful** Build the Play version **only with the live https address**. A tunnel or Wi-Fi address would stop working for every customer.

9. Publishing on the Google Play Store

9.1 The developer account

1. Go to **play.google.com/console** and sign in with a Google account the business will keep (not a staff member's personal one).
2. Choose **Organisation** as the account type if you can (needs a D-U-N-S number for Express Autocare, free from Dun & Bradstreet, takes a few days). An organisation account can publish straight to production; a new **personal** account must first run a closed test with at least 12 testers for 14 days.
3. Pay the one-time registration fee (US$25) and complete identity verification (ID, address, phone; a few days).

9.2 Create the app and the store listing

1. **Create app**: name "Express Autocare", default language English (India), App, Free; accept the declarations.
2. **Main store listing**: short description (up to 80 characters), full description, app icon 512 × 512 PNG, feature graphic 1024 × 500, and at least 2 phone screenshots (Home, Job, Book, Ask work well; take them on a phone or use the ones in the User Manual).
3. **Privacy policy**: a page on your website saying what the app stores (name, phone, cars, service history, documents and photos the customer adds, UPI references they send), why, and how to ask for deletion. Put its address in App content › Privacy policy.
4. **App content** forms: Ads (no), App access (see 9.3), Content rating questionnaire (no violence etc. → Everyone), Target audience (18+), Data safety (see 9.4), News app (no), Government app (no), Financial features (it does not process payments itself; it opens UPI apps).

Store text	Example
Short description	Track your car's service live, approve work, pay by UPI and book in a few taps.
Full description (start)	The official app of Express Autocare, multi-brand car service. See your car's service live with photos and videos, approve or decline extra work, pay by UPI, keep your car's service book, insurance and PUC, book pickup & drop or home service, and ask our assistant what that noise is.

9.3 App access for Google's reviewers

Reviewers must be able to sign in. Create a **test customer** with a car and a job, and in App content › App access give: "Sign in with mobile number 9xxxxxxxxx; the code is shown to staff — please use this sign-in link instead: https://erp.expressautocare.in/j/… (opens the app signed in)". Keep that test job open while the review runs.

9.4 Data safety answers (customer app)

Data	Collected?	Why
Name, phone number	Yes	Account, app functionality
Photos (documents, assistant, UPI screenshot)	Yes, only when the customer adds them	App functionality
Files and documents	Yes (insurance, PUC, RC)	App functionality
Financial info	UPI reference numbers only (no card or bank details)	Payment confirmation
Location	No (the customer app does not read location)	-
Shared with third parties	Questions to the assistant go to Anthropic (AI) when it is switched on	App functionality
Encryption / deletion	Encrypted in transit (https); customers can ask the workshop to delete their data	-

9.5 Testing tracks, then production

1. **Internal testing**: Testing › Internal testing › Create release › upload **app-release.aab** › add testers (email list) › roll out. Testers install from the Play link within minutes. Use this first.
2. **Closed testing** (required for new personal accounts: 12+ testers for 14 days): invite regular customers or staff family; collect feedback.
3. **Production**: Production › Create release › choose the tested bundle › release notes › **Send for review**. The first review usually takes a few days. Choose a staged rollout (for example 20%) if you want to start small.

9.6 Updates

- Changes to screens and features come from the server: **no new app needed**.
- A new app version is needed only for native changes (permissions, plugins, icon, name, target Android version). Increase **versionCode**, build (8.2) and upload a new release.

9.7 The staff app and the Play Store

The staff app does not need to be on the Play Store: share the signed APK (build:release in the **mobile** folder gives an .aab; for a shareable APK use Android Studio › Build › Generate Signed APK). If you prefer the Play Store, publish it as **unlisted** or through **Managed Google Play** (private to your organisation). Google then asks for a **foreground service declaration** for location (with a short screen recording showing punch in → "sharing your location" notification → punch out) because it shares location in the background.

> **Note** iPhone App Store: needs a Mac with Xcode and an Apple Developer account (US$99 a year). Until then iPhone users install the web apps from Safari (Add to Home Screen).

10. Problems and fixes

Problem	Fix
Phone shows "This site can't be reached" on http://192.168…	Same Wi-Fi? System started with --host 0.0.0.0? Windows firewall allows PHP / port 8000 (section 2)?
The page loads but is blank or unstyled on the phone	public\hot exists: run npm run build and delete public\hot (section 2).
"Turn on location" although it is on (browser)	The address is not https. Use the tunnel address (section 3).
APK will not install	Allow "Install unknown apps" for the app that opened it; uninstall an older copy signed with a different key first.
Staff app: "Can't reach the workshop system"	The tunnel address changed or the system is stopped: start it and use Change server address.
Location stops after some minutes with the screen locked	Battery: Unrestricted; Xiaomi Autostart on; the app not "cleared" from recent apps.
Photos do not open over the tunnel	APP_URL in .env must be the tunnel https address; then php artisan config:clear.
Customer app notifications never arrive	Needs https and the customer must allow notifications (Inbox). Windows test computers also need OPENSSL_CONF set (see docs/CUSTOMER_APP.md).
Play Console: "targets an old API level"	Raise targetSdkVersion (section 8.1) and upload a new versionCode.
Play Console: "app access" rejection	Give reviewers a working sign-in link and keep the test job open (9.3).

Note Technical notes for whoever maintains the system are in the project: docs/ANDROID_APP.md and docs/CUSTOMER_APP.md.